Manual do Modelo

Equacao de Schrodinger Estacionaria

Elmer FEM 9.0 — SchrodingerSolve

Módulo: `SchrodingerSolve`

Sub-rotina: `SchrodingerSolver`

Autor do módulo: (projeto UFPA — ver *Como citar*)

Autor do documento: (projeto UFPA)

Versão do Elmer: 9.0

Sumário

1. Introdução[1]
2. Teoria[2]
3. Condições de contorno[3]
4. Potencial V(r)[4]
5. Keywords[5]
6. Unidades e convenções[6]
7. Soluções analíticas de referência[7]
8. Limitações[8]
9. Como citar[9]

---

1. Introdução

Este módulo resolve a **equação de Schrödinger estacionária** (independe do tempo), que é o problema de autovalores fundamental da mecânica quântica:

$$-\frac{\hbar^2}{2m}\nabla^2\psi + V(\mathbf{r})\,\psi = E\,\psi$$

O módulo calcula os `N` menores autovalores `E` (as energias) e as autofunções `ψ` (as funções de onda) associadas. Serve para:

- poços de potencial (partícula em uma caixa, poço finito)

[1] \{}#1-introdução
[2] \{}#2-teoria
[3] \{}#3-condições-de-contorno
[4] \{}#4-potencial-vr
[5] \{}#5-keywords
[6] \{}#6-unidades-e-convenções
[7] \{}#7-soluções-analíticas-de-referência
[8] \{}#8-limitações
[9] \{}#9-como-citar

- osciladores harmônicos
- potenciais moleculares (Morse)
- poços duplos (tunelamento)
- potenciais periódicos (estrutura de bandas)

O módulo opera em **1D, 2D e 3D** com a mesma implementação; a dimensão é deduzida do `Coordinate System` declarado na seção `Simulation`.

> ***Nota:*** *para o problema* dependente do tempo *(equação de Schrödinger não-linear, propagação de pacotes de onda), este módulo não se aplica — ele resolve apenas estados estacionários.*

---

2. Teoria

2.1 Forma fraca

A equação é discretizada pelo **método de Galerkin** com elementos finitos. Expandindo a solução na base de funções de forma `Nᵢ`,

$$\psi(\mathbf{r}) = \sum_i N_i(\mathbf{r})\,\psi_i$$

e testando com `Nⱼ` (integrando por partes o termo laplaciano), obtém-se

$$\underbrace{\int_\Omega \nabla N_j \cdot \nabla N_i \, d\Omega}_{K_{ji}}\,\psi_i + \underbrace{\int_\Omega N_j \, V \, N_i \, d\Omega}_{M^a_{ji}}\,\psi_i = E\,\underbrace{\int_\Omega N_j N_i \, d\Omega}_{M_{ji}}\,\psi_i$$

isto é, o **autoproblema generalizado**:

$$A\,\psi = E\,M\,\psi, \qquad A = K + M^a$$

onde

Matriz	Nome	Expressão	Significado
`K`	rigidez	`∫ ∇Nᵢ·∇Nⱼ dΩ`	energia cinética
`M`	massa	`∫ NᵢNⱼ dΩ`	produto interno (normalização)
`Mᵃ`	massa ponderada	`∫ Nᵢ V Nⱼ dΩ`	energia potencial

2.2 Solução do autoproblema

O autoproblema generalizado é resolvido pela rotina **LAPACK** `DSYGV`, apropriada para matrizes simétricas com `M` definida positiva. Os `N` menores autovalores são ordenados e reportados.
A condição de contorno de Dirichlet é imposta por **eliminação dos graus de liberdade fixos**, e não pelo artifício de substituir a diagonal por 1 (que introduz autovalores espúrios).

2.3 Quadratura

As integrações usam **quadratura de Gauss** nos pontos do elemento, com `N` pontos por direção. Isso significa que o potencial `V` é avaliado nos pontos de Gauss e integrado exatamente, e não interpolado de forma aproximada pelos nós — importante para potenciais que variam rapidamente.

---

3. Condições de contorno

3.1 Dirichlet homogênea (padrão)

```
Apply Dirichlet = Logical True
```

Os nós pertencentes ao contorno geométrico da malha são detectados **automaticamente** e recebem `ψ = 0`. Isso corresponde à parede infinita (poço infinito) e é a condição adequada para a maioria dos casos ligados.
Não é necessário declarar `Boundary Condition` no SIF.

3.2 Detecção automática

O solver identifica os nós de contorno pela geometria da malha (coordenadas nos extremos do domínio em cada direção), e não pela lista de elementos de contorno. Isso torna a implementação robusta a malhas geradas externamente.

3.3 Direção da normal

O fluxo `∇ψ·n̂` é naturalmente nulo onde não há condição de Dirichlet imposta (condição de Neumann homogênea) — o que corresponde a uma parede refletora.

> ***Limitação importante:*** *este módulo* ***não implementa condição de radiação*** *(*Sommerfeld*) nem* complex scaling*. Portanto* ***não*** *serve para calcular coeficientes de transmissão através de barreiras nem estados ressonantes. Para isso seria necessário um tratamento de domínio infinito (ver* `00_planejamento/referencias/` *— o programa* `qsolver`*, CPC 329 (2026) 110393).*

4. Potencial V(r)

O potencial é lido como **campo nodal** e admite três formas, em ordem de precedência.

4.1 `Potential` em `Body Force` (recomendado)

1D — uma variável; `tx` é o valor da coordenada:

```
Body Force 1
  Potential = Variable Coordinate 1
    Real MATC "1.0*tx*tx"
End
```

2D/3D — múltiplas variáveis separadas por **vírgula**; `tx` é a primeira e `tx(1)`, `tx(2)` são as seguintes:

```
Body Force 1
  Potential = Variable Coordinate 1, Coordinate 2, Coordinate 3
    Real MATC "0.25*(tx*tx + tx(1)*tx(1) + tx(2)*tx(2))"
End
```

> ***O separador é VÍRGULA, não espaço.*** *`Variable Coordinate 1 Coordinate 2` é interpretado como **um único nome** de variável inexistente. A causa está no parser do Elmer (`Lists.F90, ListParseStrToVars`): `l1 = INDEX( str(l0:slen), ',' )`. Ver `01_simulacoes/schrodinger_P1_2/docs/SINTAXE_MATC_POTENCIAL.md`.*

4.2 `Potential X` / `Potential Y` / `Potential Z`

Construção por componentes, combinada internamente pelo solver (`V = Vₓ + V_y + V_z`). Cada termo é uma expressão 1D:

```
Body Force 1
  Potential X = Variable Coordinate 1
    Real MATC "0.25*tx*tx"
  Potential Y = Variable Coordinate 2
    Real MATC "0.25*tx*tx"
  Potential Z = Variable Coordinate 3
    Real MATC "0.25*tx*tx"
End
```

Útil quando os termos vêm de fontes diferentes ou quando se quer evitar o MATC multidimensional.

4.3 Ausente

Se nenhuma das formas acima estiver presente, `V(r) = 0` (poço infinito).
O solver **informa em log** qual caminho foi usado, para não haver leitura silenciosa de `V = 0`:

```
SchrodingerSolver: V(x) lido como campo nodal de Body Force.
SchrodingerSolver:   V_min =   0.000000E+00   V_max =   5.000000E+01
```

> *O log de `V_min/V_max` é a verificação mais rápida de que o potencial foi lido corretamente. Ele é reportado **por componente** quando se usa a forma `Potential X/Y/Z`, e somado internamente.*

---

5. Keywords

5.1 Seção `Simulation`

Keyword	Tipo	Descrição
`Coordinate System`	String	`Cartesian 1D`, `Cartesian 2D` ou `Cartesian 3D`. **Deve corres**
`Simulation Type`	String	`Steady State`
`Steady State Max Iterations`	Integer	`1`
`Max Output Level`	Integer	Use **≥ 4** para ver as mensagens do solver (ele reporta e

5.2 Seção `Body Force`

Keyword	Tipo	Descrição
`Potential`	Real (campo)	Potencial `V(r)`. Campo nodal ou expressã
`Potential X`, `Potential Y`, `Potential Z`	Real (campo)	Componentes do potencial. Ver §4.2.

5.3 Seção `Solver`

Keyword	Tipo	Padrão	Descrição
`Equation`	String	—	Nome da equação (ex.: `"Schrodinger"`)
`Procedure`	String	—	`"SchrodingerSolver" "SchrodingerSolver"`
`Variable`	String	`psi`	Nome do campo de saída (a função de onda)
`Variable DOFs`	Integer	`1`	Graus de liberdade por nó. Manter em 1.
`Eigen System Values`	Integer	`5`	**Número de autovalores a calcular** (as `N` me

Keyword	Tipo	Padrão	Descrição
`Apply Dirichlet`	Logical	`True`	Impõe $\psi = 0$ nos nós de contorno (§3)
`Linear System Solver`	String	`Direct`	Manter `Direct`
`Linear System Direct Method`	String	—	`"umfpack"` ou `"mumps"` (se a build tiver MUMPS)
`Exportar Matrizes`	Logical	`False`	Grava `A` e `M` em triplas ASCII (§5.4)
`Exportar Prefixo`	String	`schrod`	Prefixo dos arquivos exportados

5.4 Exportação de matrizes (`Exportar Matrizes`)

Quando `Exportar Matrizes = Logical True`, o solver grava:

Arquivo	Conteúdo	Formato
`<prefixo>_A.dat`	operador $A = K + M^a$	`n n nnz` e depois `i j valor` (**0-based**)
`<prefixo>_M.dat`	matriz de massa `M`	idem
`<prefixo>_fixed.dat`	GDL de Dirichlet	contagem e depois os índices (**0-based**)

Para que serve: permite resolver o autoproblema **fora do Elmer**, com solver esparso (`scipy.sparse.linalg.eigsh`, ARPACK shift-invert) ou paralelo (PETSc/SLEPc). Isso é necessário porque o `DSYGV` é **denso e serial** (memória $O(n^2)$, custo $O(n^3)$) e porque o Elmer 9.0 **não consegue particionar malha** nesta plataforma.

Resolver o que foi exportado:

```
python3 01_simulacoes/schrodinger_P1_2/scripts/resolve_exportado.py schrod 5
```

Ver `01_simulacoes/schrodinger_P1_2/docs/PARALELISMO.md` para a análise completa.

5.5 Saída VTU

Para gerar o arquivo `.vtu` (ParaView), acrescente um segundo solver:

```
Solver 2
  Exec Solver = "After Simulation"
  Equation = "ResultOutput"
  Procedure = "ResultOutputSolve" "ResultOutputSolver"
  Output File Name = File "run"
  Output Format = String "vtu"
End
```

E ative ambos: `Active Solvers(2) = 1 2`.
O arquivo sai em **`malha/run_t0001.vtu`** (o Elmer escreve no diretório da malha) e contém a malha + o campo **`psi`** (autofunção do **modo 1**).

---

6. Unidades e convenções

O solver trabalha com

$$\hbar = 1, \qquad m = 1$$

de modo que a equação implementada é

$$-\nabla^2 \psi + V(\mathbf{r})\,\psi = E\,\psi$$

Relação com a forma usual

A forma que aparece em muitos textos (e no manual do Elmer para o `HelmholtzSolver`) é

$$-\tfrac{1}{2}\nabla^2 \psi + V\psi = E\psi$$

Esta é diferente. A conversão é:

$$E_{\text{usual}} = \tfrac{1}{2}\,E_{\text{aqui}} \qquad \Longleftrightarrow \qquad E_{\text{aqui}} = 2\,E_{\text{usual}}$$

Em unidades atômicas, o solver adota efetivamente `m* = 1/2`, ou seja, o **dobro** da massa do elétron. Para converter a um sistema físico concreto, use a expressão completa

$$E = \frac{\hbar^2}{2m} \times E_{\text{adimensional}}$$

com o `E_adimensional` reportado pelo solver.

> ***Por que isso importa:*** *a validação inicial do projeto usou o poço infinito com `L = 40`, onde **as duas convenções coincidem por acidente numérico** (`π²/L²` vs `π²/2L²` só diferem por fator 2, mas o teste não distinguia porque o valor comparado era o mesmo). A convenção só foi determinada com testes que **distinguem** as formas: poço infinito com `L = 10` e oscilador harmônico. Ver `01_simulacoes/schrodinger_P1_2/docs/VALIDACAO_POTENCIAL.md`.*

---

7. Soluções analíticas de referência

Fórmulas **na convenção deste solver** (`−∇²ψ + Vψ = Eψ`). Todas verificadas numericamente (ver `01_simulacoes/RESULTADOS_TODOS_CASOS.md`).

Potencial	Autovalores	Observação
Poço infinito 1D, `[0,L]`	$E_n = (n\pi/L)^2$, $n = 1,2,\ldots$	
Poço infinito `d`-dimensional, lado `L`	$E = (\pi^2/L^2)\Sigma_i\, n_i^2$	degeneresc
Oscilador harmônico isotrópico $V = \alpha\, r^2$	$E_n = 2\sqrt{\alpha}\,(n + d/2)$	degeneresc
Potencial de Morse	mínimo do poço $= -D_e$ (verificação de `V`)	energia de
Poço duplo $V = a(x^2-b)^2$	splitting $E_1-E_0 > 0$	mede tunel
Potencial periódico $V = A\cos(kx)$	bandas	ver §8

---

8. Limitações

Registradas explicitamente para evitar uso indevido.

Limitação	Detalhe
Domínio finito	É necessário truncar o domínio com $\psi = 0$. Estados excitados de
Sem condição de radiação	Não calcula transmissão/reflexão por barreiras nem estados res
Denso e serial	`DSYGV` é $O(n^2)$ em memória e $O(n^3)$ em custo. Na prática: ~8000
Sem MPI	O solver **não é MPI-aware**: monta matrizes **densas locais**. En
Só o modo 1 no VTU	Tentativas de publicar todos os modos via `VariableAdd` causaram
Convenção `m = 1`	Ver §6.

---

9. Como citar

```
SchrodingerSolve — módulo de elementos finitos para a equação de Schrödinger
estacionária, Elmer FEM 9.0. Projeto NLSE/Schrödinger, UFPA.
```

Documentos relacionados:

- `Manual_Solver_Schrodinger.md` — implementação, compilação e uso
- `01_simulacoes/schrodinger_P1_2/docs/` — validação e sintaxe
- `01_simulacoes/schrodinger_P1_2/casos/<caso>/TUTORIAL.md` — tutoriais

Última atualização: 08/10/2026